



3 DE OCTUBRE DE 2026

OPTIMIZACIÓN DE EMPAQUE DE PRODUCTOS MEDIANTE ARQUITECTURA MIPS

ALEX ANDRES NARANJO ORDOÑEZ
DIMAS DANIEL BORBOR POZO

UCOM250_81
ORGANIZACION Y ARQUITECTURA DE COMPUTADORES
2026



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
Optimización de Empaque de Productos mediante Arquitectura
MIPS

Contenido

Descripción.....	2
Análisis.....	2
Implementación	2
Conclusiones	6
Bibliografía	6

Descripción

Una empresa tiene productos que deben colocarse en cajas de igual capacidad, teniendo una cantidad inicial de productos y de capacidad, el problema aquí reside en que no todas las cajas están completas, y la manera de solucionarla es separando las cajas completas con los productos que sobran imprimiendo los productos sobrantes

Análisis

A continuación, se identifican los datos proporcionados y almacenados en memoria dentro del escenario planteado, así como los resultados (ver Tabla 1)

Tabla 1 Datos del programa

Datos	Propósito	Operación requerida	Instrucción MIPS
Dato 1	Representa la cantidad total de productos (125 productos).	Carga	lw
Dato 2	Representa la capacidad máxima de cada caja (12 unidades).	Carga	lw
Resultado 1	Calcula el número de cajas completas empaquetadas (10 cajas).	División	div
Resultado 2	Calcula la cantidad de productos sobrantes sin empacar (5 productos).	Residuo	rem
Resultado 3	Verifica si la cantidad de productos y capacidad son equivalente (0 = Falso).	Comparación	seq

Implementación

La solución fue implementada en el lenguaje ensamblador MIPS. A continuación se presenta el código fuente completo y funcional:

```
.data
dato1:      .word 125 #productos
dato2:      .word 12  #capacidad
resultado1: .word 0  #cajas completas
```



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
Optimización de Empaque de Productos mediante Arquitectura
MIPS

```
resultado2: .word 0 #producto sobrante
resultado3: .word 0 #coincidencia entre cajas y productos

mensaje_productosresiduo: .asciiz "Productos sin empacar: "
#preparamos el mensaje desde la memoria
mensaje_productoscom: .asciiz "Empaque completo" #se prepara el
mensaje desde la memoria

.text
.globl main

main:
    lw $t0, dato1 #cargamos el valor al registro t0 desde la memoria
    al registro
    lw $t1, dato2 #cargamos el valor al registro t1 desde la memoria
    al registro

    div $t2, $t0, $t1 #operacion para calcular cajas completas
    rem $t3, $t0, $t1 #operacion para calcular el sobrante de los
    productos
    seq $t4, $t0, $t1 #pregunta si coincide los productos y las
    cajas

    sw $t2, resultado1 #guardamos la cantidad de cajas completas en
    resultado 1 del registro a la memoria
    sw $t3, resultado2 #guardamos la cantidad de producto sobrante
    en resultado 2 del registro a la memoria
    sw $t4, resultado3 #guardamos si salio 1 o 0 en resultado 3 para
    ver la coincidencia

    bne $t3, $zero, sobrante #si residuo != 0 salta a la etiqueta
    sobrante, si no, continua

    li $v0, 4 #syscall 4 nos dice que se imprime caracteres
    la $a0, mensaje_productoscom #cargamos la direccion del mensaje
    en $a0
    syscall #se imprime el mensaje

    li $v0, 0 #finalizacion del programa
    jr $ra

sobrante:
    li $v0, 4 #syscall 4 nos dice que se imprime caracteres
```

```

la $a0, mensaje_productosresiduo #cargamos la direccion del
mensaje en $a0
syscall #se imprime el mensaje

li $v0, 1 #syscall 1 nos dice que se imprime un entero
add $a0, $t3, $zero #movemos $t3 a $a0 con la ayuda de sumando
$zero
syscall #se imprime el entero en este caso el residuo

li $v0, 10 #finalizamos el programa
syscall

```

Explicación de los bloques principales del programa:

1. Declaración de datos (.data): Se definen los datos de entrada (125 productos y capacidad de 12 por caja), las variables para almacenar los resultados en memoria y las cadenas de texto para la salida por consola.

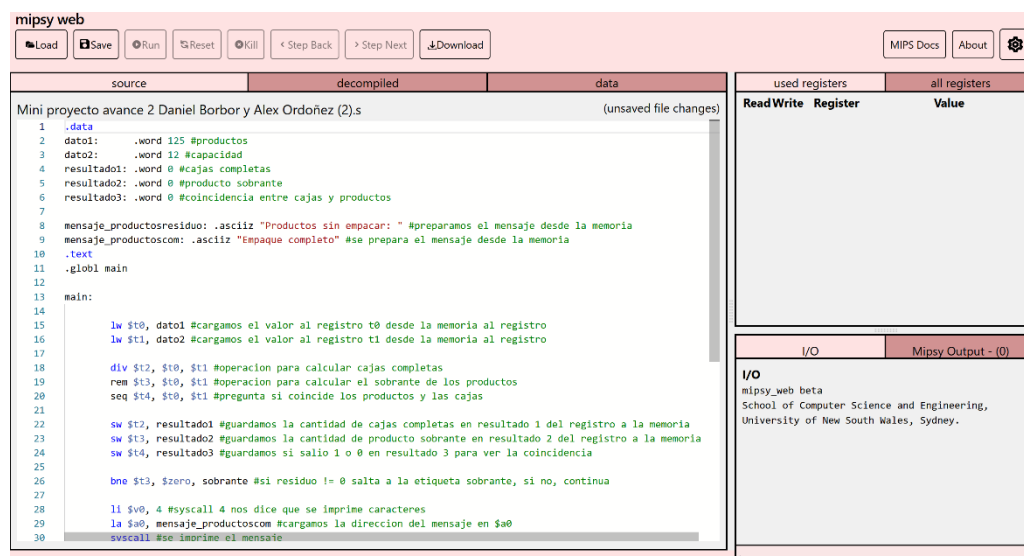


Figura 1. Estructura general del código fuente en el simulador MIPS.

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

FACULTAD DE INGENIERÍA

Optimización de Empaque de Productos mediante Arquitectura MIPS

2. Carga y Cálculo (.text / main): Mediante lw se cargan los datos en los registros \$t0 y \$t1. Luego se efectúa la división entera (div) para obtener las cajas completas (\$t2), el residuo (rem) para los productos sobrantes (\$t3) y la comparación de igualdad (seq) para verificar coincidencia (\$t4).

source	decompiled	data
Mini proyecto avance 2 Daniel Borbor y Alex Ordoñez (2).s		
Data segment		
0x10010000	7d 00 00 00 0c 00 00 00 0a 00 00 00 05 00 00 00	} \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
0x10010010	00 00 00 00 50 72 6f 64 75 63 74 6f 73 20 73 69	\0 \0 \0 \0 P r o d u c t o s s i
0x10010020	6e 20 65 6d 70 61 63 61 72 3a 20 00 45 6d 70 61	n e m p a c a r : \0 E m p a
0x10010030	71 75 65 20 63 6f 6d 70 6c 65 74 6f 00 00 00 00	q u e c o m p l e t o \0 _ _ _

Figura 2. Inspección del segmento de datos (memory data segment) en la ejecución.

3. Almacenamiento: Se guardan los resultados en memoria con sw. Posteriormente, la instrucción bne verifica si el residuo en \$t3 es diferente de cero. De existir residuos, salta a la etiqueta sobrante para imprimir el mensaje correspondiente y el valor entero del residuo mediante llamadas al sistema (syscall).

used registers	all registers
ReadWrite Register	Value
⊙ ⊙ \$zero	0
⊙ ⊙ \$at	0x10010010
⊙ ⊙ \$v0	10
⊙ ⊙ \$a0	5
⊙ ⊙ \$a1	0
⊙ ⊙ \$t0	125
⊙ ⊙ \$t1	12
⊙ ⊙ \$t2	10
⊙ ⊙ \$t3	5
⊙ ⊙ \$t4	0

I/O	Mipsy Output - (0)
I/O Productos sin empacar: 5 Program exited with exit status 0	

Figura 3. Estado final de los registros y consola de salida I/O.



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA
Optimización de Empaque de Productos mediante Arquitectura
MIPS

Conclusiones

A través del desarrollo de este proyecto se comprendió de manera práctica la lógica del procesamiento a nivel de arquitectura de computadores, comprendiendo cómo las instrucciones básicas de un conjunto de instrucciones (ISA) como MIPS permiten construir algoritmos de control de flujo e interpolación de datos de memoria.

Además, la experiencia permitió fortalecer habilidades en el análisis de problemas y su abstracción en instrucciones de bajo nivel, reconociendo la importancia del manejo adecuado de registros, llamadas al sistema (syscalls) y condicionales para lograr una ejecución eficiente y estructurada.

Bibliografía

Sánchez, C. (24 de enero de 2020). Citas APA. Normas APA. <https://normas-apa.org/citas/>

School of Computer Science and Engineering, UNSW Sydney. (2026). COMP1521 — MIPS Instruction Set Reference & Mipsy Web. University of New South Wales.
<https://cgi.cse.unsw.edu.au/~cs1521/current/resources/mips-guide.html#registers>

Wikibooks. (17 de septiembre de 2023). MIPS Assembly/Instruction Formats. Wikibooks, The Free Textbook Project. https://en.wikibooks.org/wiki/MIPS_Assembly/Instruction_Formats

Wikibooks. (29 de mayo de 2024). MIPS Assembly/Register File. Wikibooks, The Free Textbook Project. https://en.wikibooks.org/wiki/MIPS_Assembly/Register_File

Patterson, D. A., & Hennessy, J. L. (2014). *Computer Organization and Design: The Hardware/Software Interface* (5th ed., pp. 62–66). Morgan Kaufmann